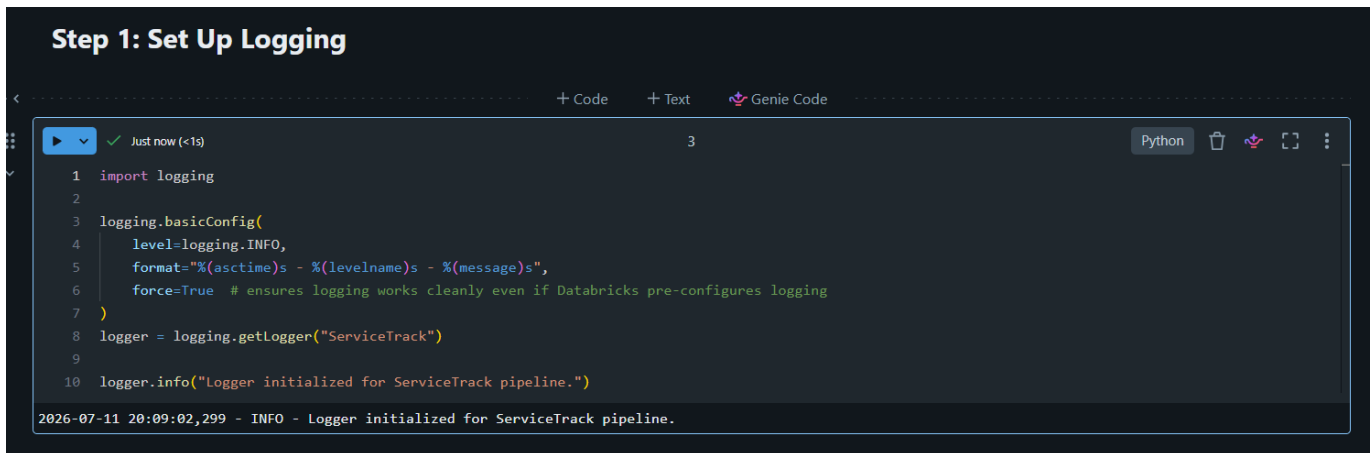


Config & Utils — Modularization, Logging & Parameterization

ServiceTrack Project | Execution Screenshots

Screenshot 1: Logging Setup



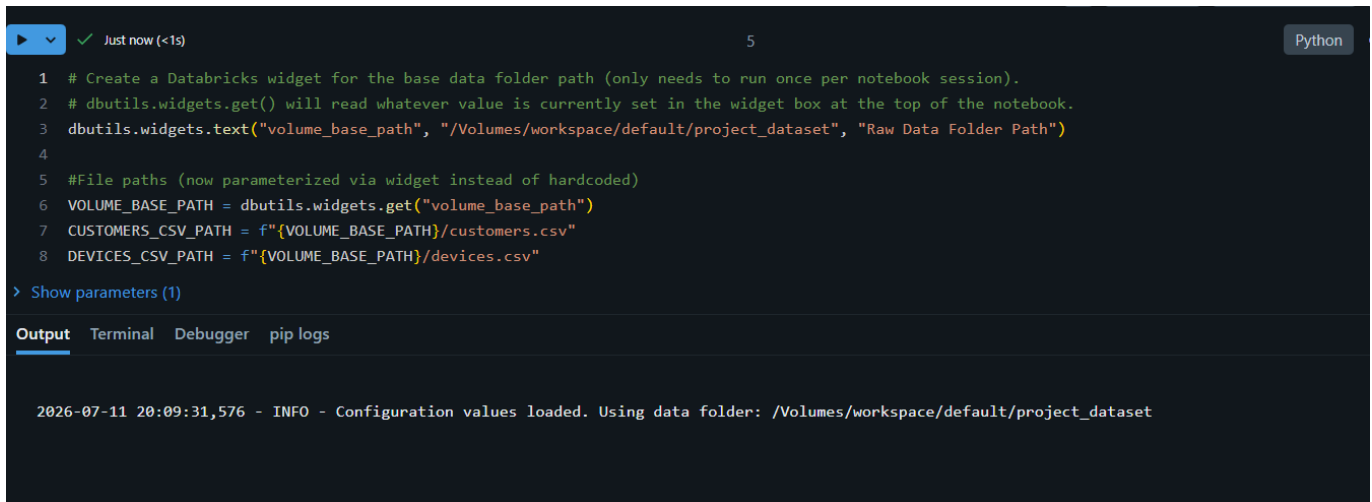
The screenshot displays a code editor interface with a dark theme. At the top, a tab is labeled "Step 1: Set Up Logging". Below the tab, the code editor shows Python code for configuring logging. The code includes imports, basic configuration with INFO level and a specific format, and an initialization message. To the right of the code editor, a "Python" button and icons for file operations are visible. Below the code, the execution output is shown, indicating a successful run with an INFO message.

```
1 import logging
2
3 logging.basicConfig(
4     level=logging.INFO,
5     format="%(asctime)s - %(levelname)s - %(message)s",
6     force=True # ensures logging works cleanly even if Databricks pre-configures logging
7 )
8 logger = logging.getLogger("ServiceTrack")
9
10 logger.info("Logger initialized for ServiceTrack pipeline.")
```

2026-07-11 20:09:02,299 - INFO - Logger initialized for ServiceTrack pipeline.

Python's logging module configured with timestamp and severity level, replacing plain print() statements.

Screenshot 2: Parameterized Configuration via Databricks Widget



The screenshot shows a Databricks notebook interface. At the top, there's a status bar with a play button, a checkmark, the text "Just now (<1s)", a cell number "5", and a "Python" language selector. The main area contains a code cell with the following Python code:

```
1 # Create a Databricks widget for the base data folder path (only needs to run once per notebook session).
2 # dbutils.widgets.get() will read whatever value is currently set in the widget box at the top of the notebook.
3 dbutils.widgets.text("volume_base_path", "/Volumes/workspace/default/project_dataset", "Raw Data Folder Path")
4
5 #File paths (now parameterized via widget instead of hardcoded)
6 VOLUME_BASE_PATH = dbutils.widgets.get("volume_base_path")
7 CUSTOMERS_CSV_PATH = f"{VOLUME_BASE_PATH}/customers.csv"
8 DEVICES_CSV_PATH = f"{VOLUME_BASE_PATH}/devices.csv"
```

Below the code cell, there's a link "> Show parameters (1)". At the bottom, there's an "Output" tab selected, showing a log message:

```
2026-07-11 20:09:31,576 - INFO - Configuration values loaded. Using data folder: /Volumes/workspace/default/project_dataset
```

Base data folder path now read from a Databricks widget instead of being hardcoded, allowing the pipeline to run on a different dataset without editing code.

Screenshot 3: Reusable Function - validate_row_count()

Step 6: Reusable Function - Validate Row Counts

```
▶ ✓ 01:26 AM (<1s) 13
1 def validate_row_count(actual_count: int, expected_count: int, label: str):
2     """
3     Compare an actual row count against an expected value and log a Success or Warning message.
4     Returns True if the check passed, False otherwise (so calling code can react if needed).
5     """
6     if actual_count == expected_count:
7         logger.info(f"Success: {label} row count matched expected value ({actual_count}).")
8         return True
9     else:
10        logger.warning(f"Warning: {label} row count was {actual_count}, expected {expected_count}.")
11        return False
```

A shared, reusable function that checks row counts against expected values and logs a Success or Warning message.